

REFACTOR ARQUITECTURA + ENTORNO LOCAL — OficioYa

Necesito que refactorices OficioYa para que tenga una arquitectura más profesional, desacoplada y preparada para desarrollo real.

Actualmente la aplicación depende demasiado de Supabase online y no puede correr correctamente en local sin variables privadas.

Eso es un problema.

OBJETIVOS PRINCIPALES

1. La app debe poder correr completamente LOCAL sin Supabase real.
 2. Debe existir un modo demo/mock robusto.
 3. La arquitectura debe desacoplar frontend de backend.
 4. El proyecto debe verse y sentirse profesional para desarrollo real.
 5. Reducir complejidad innecesaria de la POC.
-

PROBLEMAS ACTUALES A RESOLVER

1. Dependencia fuerte de Supabase

Actualmente:

- si no existe `.env`
- o no existe URL de Supabase
- la app rompe

Eso NO debe pasar.

La app debe:

- detectar automáticamente ausencia de backend
 - entrar en modo demo/local automáticamente
 - seguir funcionando completamente
-

2. Crear arquitectura híbrida (REAL + MOCK)

Necesito:

Modo demo/local

Usar:

- mock data local
- fake auth
- fake requests
- fake professionals

Modo producción

Usar:

- Supabase real
-

3. Crear capa de abstracción de datos

NO quiero llamadas directas a Supabase repartidas por toda la app.

Crear:

- services/
- repositories/
- adapters/

Ejemplo:

- professionalService
- requestService
- authService

Cada servicio debe poder:

- usar mock local
- o Supabase real

sin romper componentes.

4. Variables de entorno

Mejorar manejo de `.env`

Crear:

- `.env.example`
- validación de variables
- fallbacks seguros

Nunca romper la app si faltan variables.

5. Modo desarrollo profesional

Necesito:

- app funcionando offline/demo
 - datos mock realistas
 - UX completa aunque no exista backend
 - posibilidad de mostrar la app sin internet
-

6. Simplificar complejidad innecesaria

Analiza:

- features redundantes
- componentes innecesarios
- lógica sobrecomplicada
- estados duplicados
- flows innecesarios

Reducir complejidad manteniendo:

- experiencia premium
- UX moderna
- funcionalidades clave

La POC debe ser:

- simple
- sólida
- rápida
- clara

- demostrable
-

7. Arquitectura frontend profesional

Refactorizar:

- separación UI / lógica
- hooks reutilizables
- stores limpios
- tipado estricto
- componentes desacoplados

Evitar:

- lógica mezclada con UI
 - fetches directos en componentes
 - dependencias acopladas
-

8. Objetivo final

Quiero que OficioYa:

- pueda correrse fácilmente por cualquier desarrollador
- funcione localmente sin configuración compleja
- tenga arquitectura limpia
- esté preparada para crecer
- mantenga UX premium
- se sienta como una startup real

No quiero una demo frágil. Quiero una base sólida y profesional.